

App Dev II Project Report

1. Student Details

Name: Kanchan

Roll Number: 24f1002360

Email: 24f1002360@ds.study.iitm.ac.in

About Me: I am a Student at IIT Madras BS Degree program with a deep interest in Software Development, web application development and data driven technologies. I enjoy building meaningful applications that combine learning, analytics, and user experience.

2. Project Details

Project Title: Trekking Management System

Problem Statement:

To design and build a web based application that allows users to **Admin, Trek Staff and Trekkers to manage trekking events**, such as trek bookings and staff supervising. The system should provide role based access control, asynchronous background jobs, scheduled reminders, report generation, and API optimization using Redis caching.

Approach:

The application follows a REST API architecture using Flask as the backend and Vue.js as the frontend. SQLAlchemy ORM is used for database management, JWT for authentication, Celery with Redis for asynchronous task execution, Redis caching for performance optimization, and Bootstrap for responsive UI development.

3. AI/LLM Declaration

AI tools such as **ChatGPT** were used for assistance in code structuring, debugging suggestions, and documentation formatting. The extent of AI/LLM usage is around **15-20%**, limited to **code suggestions and documentation formatting**.

The core logic, implementation, and integration of the application were developed manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Vue.js 3	Frontend SPA and client side reactivity
Bootstrap 5	Responsive UI
Flask-JWT-Extended	JWT Authentication
SQLite	Lightweight local database for storing user data
Werkzeug	Password hashing
Celery	Background Job Processing
Redis	Celery Broker, Result Backend and API Cache
Vue Router	Client side Routing
Vite	Frontend Build Tool and Development Server
Flask-Mail	Sending Email Notifications
Flask-CORS	Cross-Origin Resource Sharing

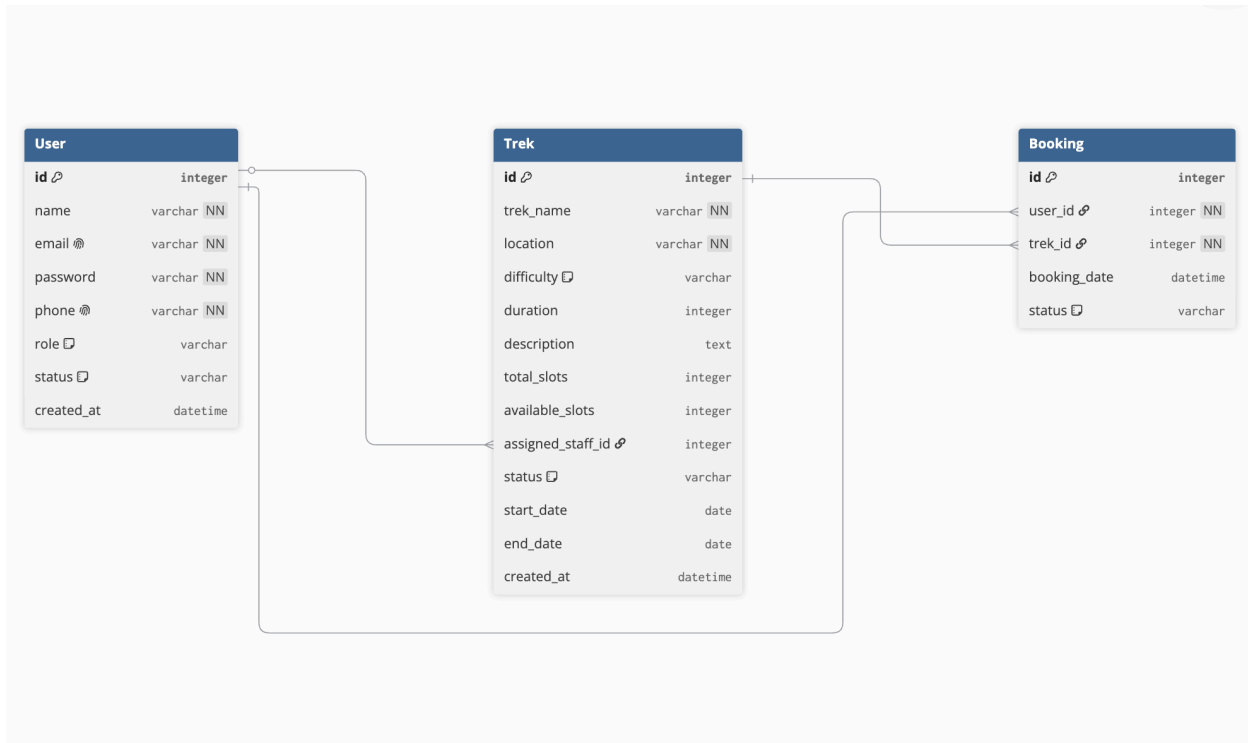
5. Database Schema / ER Diagram

Tables:

1. **User** - Stores all users of the system and their credentials. (admin_id, name, email, password, phone, role (ADMIN / STAFF / TREKKER), status, created_at)
2. **Trek** - Stores trekking event details. (id, name, location, difficulty, duration, description, total_slots, available_slots, assigned_staff_id, status, start_date, end_date, created_at)
3. **Booking** - Stores trek booking records. (id, booking_date, booking_status, trek_id, user_id)

Relationships:

- One-to-Many → Staff → Trek
- One-to-Many → Trek → Booking
- One-to-Many → Trekker → Booking



6. API Resource Endpoints

Endpoint	Method	Description
/register	POST	Register a new Trekker
/login	POST	Authenticate user and return JWT
/admin/dashboard	GET	View admin dashboard
/admin/staff	GET, POST,PUT	View Staff details, add staff and update staff
/admin/staff/<int:id>/status	PUT	For changing staff status
/admin/bookings	GET	View Booking details of users

/admin/user	GET	View user details and update status
/admin/user<int:id>	PUT	
/status		
/staff/dashboard	GET	View staff dashboard, treks and participants
/staff/treks		
/staff/participants		
/staff/treks/<int:id>	PUT	Update trek details like status and slots
/treks	GET, POST, PUT, DELETE	For admin to view , add , edit and delete treks
/treks/<int:id>		
/trekker/dashboard	GET	For view trekker dashboard , available treks, Bookings, booking history and profile
/trekker/treks		
/trekker/bookings		
/trekker/history		
/trekker/profile		
/trekker/export-history	POST	For trekkers to export their booking records
/trekker/book/<int:id>	PUT	Booking Treks, cancel treks and update trekker profile
/trekker/bookings/<int:id>		
/trekker/profile		

7. Architecture and Features

Architecture Overview

- **backend/app.py** - main Flask application entry point
- **backend/models/** - database models using SQLAlchemy
- **backend/routes/** - Flask Blueprints for admin, staff and trekker routes
- **backend/templates** - Html templates for reminder mails and monthly report
- **backend/config.py** - for database, JWT, redis, celery, and cache configuration
- **backend/tasks** - Celery backend jobs
- **frontend/src/views** - UI vue.js pages for admin staff and trekkers

- **frontend/src/router/** - Defines client side routing using Vue Router.
- **frontend/src/components/** - Reusable Vue components used across different pages.
- **frontend/App.vue** - Root Vue component that loads the entire Single Page Application.

Implemented Features

- Trekker registration and login along with dashboard
- Admin , Staff login and dashboard
- Admin can create and manage Treks and assign staff
- Trekker/users can book available Treks, cancel and view their bookings
- Staff can manage Treks status and Trek slots
- Responsive UI built with Bootstrap
- Secure password hashing and user session handling
- Daily reminder emails for trekkers
- Monthly report for admin
- Export booking history records for trekkers
- JWT token authentication

Additional Features

- Prevention of overbooking
- Booking cancellation
- Staff assignment and conflict checking
- Search and filtering
- Role based authorization

8. Video Presentation

Drive Link:

https://drive.google.com/file/d/1YG5Blr7LyfpmWGcVzBbUL6XtL_R2w3em/view?usp=sharing
